Pasillo — Plan de construcción

Codex Community Hackathon GDL · Oct 28, 2026 · @Sebastian Gandoulf Perez

Resumen

Pasillo es el companion de networking para conferencias: llenas tu perfil una vez, conectas con alguien escaneando su QR o con un @tag, y al final ambos reciben un resumen de lo que platicaron. Todo se ve en vivo en un dashboard estilo Kahoot con leaderboard y los temas más hablados.

Pitch en una línea: convierte el pasillo de una conferencia en datos, sin que nadie tenga que sacar LinkedIn.

Por qué puede ganar:

- **El público es el usuario.** En JSConf la gente escanea el QR de la pantalla, se conecta entre sí y ve el podio y los temas moverse con sus propias acciones. Votar y usar la app son lo mismo.
- **Dolor real con cliente claro.** Al asistente se le olvida a quién conoció y no da seguimiento; al organizador le falta saber de qué habla de verdad su comunidad.
- **Uso significativo de Codex.** Construido con hilos en paralelo en worktrees, un `AGENTS.md` como contrato y una skill, documentado en el README.

Regla de diseño: de abrir la app a conexión hecha en 2 toques, sin cuenta, sin instalar nada y sin login.

Problema, usuarios y objetivos

El objetivo de esta noche es un flujo completo y demostrable de punta a punta: perfil → conexión → resumen → puntos → dashboard en vivo.

Problema

- En una conferencia conoces a mucha gente y al final no recuerdas quién era quién ni de qué hablaron.
- Intercambiar contacto es lento (buscar en LinkedIn, deletrear usuarios) y casi nadie da seguimiento.
- El organizador mide asistencia y check-ins, pero no sabe qué le interesa de verdad a su comunidad ni quién conectó con quién.

Usuarios

Usuario	Qué necesita	Qué recibe
Asistente	Conectar rápido y recordar	Tarjeta del otro con sus redes y resumen de la plática
Organizador	Saber qué pasa en su evento	Dashboard en vivo con temas, conexiones y leaderboard
Público en el escenario (demo)	Entender en 60 s	Podio y temas moviéndose con sus acciones

Métricas de éxito para la demo

- Un asistente nuevo crea perfil y conecta en menos de 60 segundos.
- El dashboard `/live` se actualiza en menos de 2 segundos tras cada conexión o resumen.
- Durante la demo, al menos 20 personas del público se conectan entre sí.

Alcance del MVP

El MVP tiene tres piezas: perfil con redes, conexión con resumen, y dashboard gamificado en vivo. La app se usa solo durante la conferencia.

Dentro (P0 = no se recorta)

#	Funcionalidad	Prioridad
1	Perfil por formulario (una vez), con enlaces a redes opcionales	P0
2	Handle + código de recuperación (sin login ni contraseña)	P0
3	QR personal; escanear con la cámara nativa crea la conexión	P0
4	Tarjeta del otro con botones "Seguir en…"	P0
5	Resumen de la interacción escrito o dictado, aprobado y enviado a ambos	P0

#	Funcionalidad	Prioridad
6	Dashboard `/live`: leaderboard, temas en vivo, contadores	P0
7	Datos simulados (~150 asistentes) para que `/live` nunca se vea vacío	P0
8	Conexión por @tag con autocompletado	P1
9	Grabar la conversación con consentimiento del otro	P1
10	Rondas del leaderboard, insignias, tribus vs. tribus	P2
11	"Pregúntale a tu evento" con Codex SDK	P2

Fuera de alcance

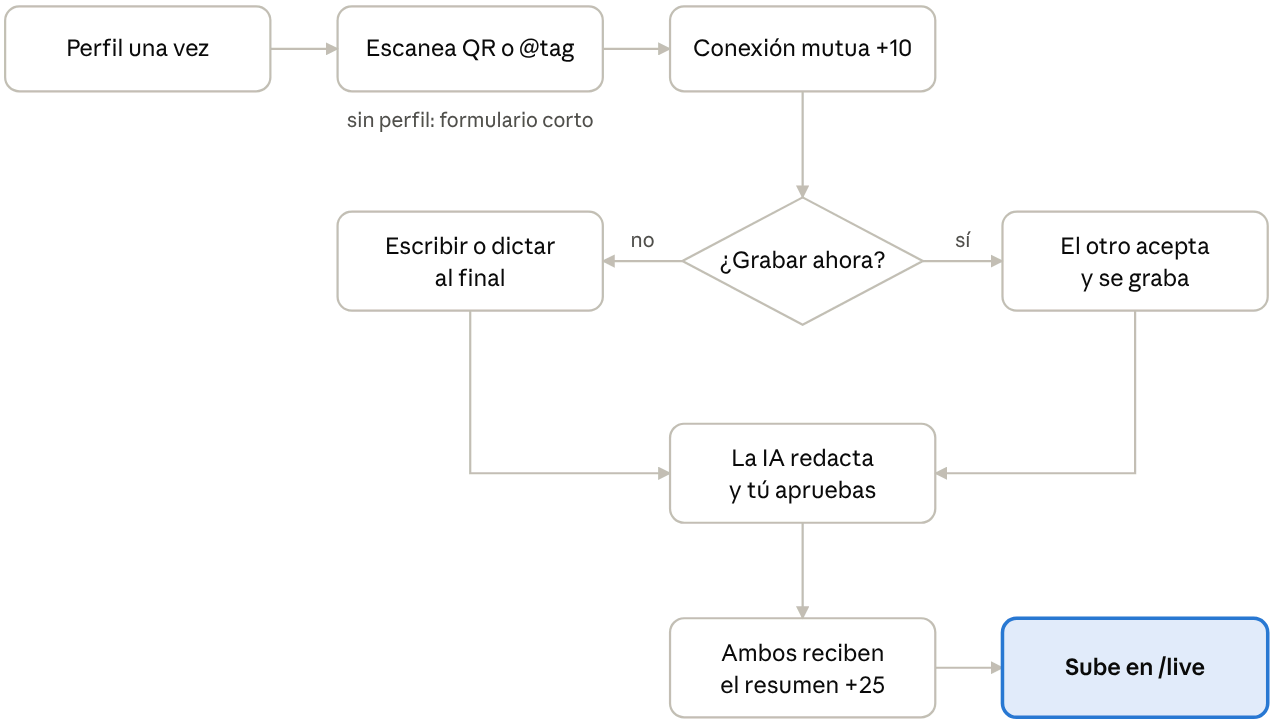
- Login con OAuth (GitHub, LinkedIn, Instagram, etc.). Las redes son solo enlaces para que te sigan.
- App nativa o instalación como PWA.
- Modo recap posterior al evento, correos y exportaciones.
- Matchmaking o recomendaciones de a quién conocer.

Orden de recorte si falta tiempo: primero P2, luego la grabación, luego el @tag. Nunca se recortan el QR, el resumen ni `/live`.

Flujo de usuario

Conectar toma dos toques; grabar es opcional y, si nadie graba, el resumen se escribe o dicta al final. Ambos caminos pasan por tu aprobación antes de llegar a los dos.

Conectar toma dos toques y el resumen llega a ambos



flujo de conexión · 8 pasos, 1 decisión

Si quien escanea no tiene perfil, llena el formulario corto y queda conectado al guardar.

Specs por pantalla

Son siete pantallas; cada una tiene criterios de aceptación que sirven como checklist de QA del PR.

1. Perfil `/join`

- Campos: **nombre (obligatorio)**, foto, empresa, rol, intereses (chips de un vocabulario fijo), busco, ofrezco, tribu.
- Redes (todas opcionales): LinkedIn, GitHub, X, Instagram, Facebook, TikTok, sitio web. Aceptan `@usuario` o URL completa y se normalizan a URL.
- Opción "Dictar mi perfil": voz → transcripción → el formulario se pre-llena y el usuario confirma.
- Al guardar: se genera handle único (`@ana`) y código de 3 caracteres (`7K2`). La sesión se guarda en cookie.

Criterios de aceptación:

- [ ] Con solo el nombre se puede guardar.
- [ ] `@ana`, `ana` y `https://x.com/ana` producen la misma URL normalizada.
- [ ] Handle duplicado → se agrega un sufijo numérico automáticamente.
- [ ] Handle + código recuperan la sesión en otro navegador.

2. Inicio `/me`

- QR grande (codifica `https://<dominio>/c/<handle>`), con handle y código debajo.
- Buscador `@` para conectar con alguien (P1).
- Contador propio: conexiones y puntos.

Criterios de aceptación:

- [ ] El QR se escanea con la cámara nativa de iPhone y Android y abre la URL correcta.
- [ ] La pantalla carga en menos de 1 s en 4G.

3. Escaneo `/c/[handle]`

- Con sesión: crea la conexión mutua y lleva a la pantalla 4.
- Sin sesión: muestra la tarjeta de quien te invitó y el formulario corto de `/join`; al guardar, conecta automáticamente.

Criterios de aceptación:

- [ ] Escanear dos veces el mismo QR no duplica la conexión ni los puntos.
- [ ] Escanear tu propio QR muestra un mensaje y no conecta.

4. Conexión hecha `/connection/[id]`

- Mensaje "Conectado con Ana · +10 puntos" y la tarjeta del otro con botones "Seguir en…" que abren cada red.
- Dos acciones: **Grabar** (P1) o **Resumir después**.

Criterios de aceptación:

- [ ] Ambos ven la tarjeta del otro en menos de 2 s.
- [ ] Cada botón de red abre la URL correcta en pestaña nueva.

5. Resumen `/connection/[id]/summary`

- Entrada por texto o dictado (o la grabación, si se hizo).
- La IA devuelve resumen, temas y siguientes pasos; el usuario edita y **aprueba con un toque**.
- Al aprobar, el resumen aparece en la conexión de ambas personas.

Criterios de aceptación:

- [ ] Un resumen vacío o sin contenido real no se puede aprobar.
- [ ] Ambos ven el resumen aprobado; solo el autor puede editarlo.

6. Mis conexiones `/connections`

- Lista de personas con foto, nombre, empresa, redes y estado del resumen (pendiente / enviado).

Criterios de aceptación:

- [ ] Las conexiones con resumen pendiente aparecen primero.

7. Dashboard `/live`

Especificado en la sección de gamificación.

Gamificación y dashboard `/live`

`/live` es la pantalla que se proyecta: un podio estilo Kahoot, los temas más hablados y contadores, actualizados en tiempo real.

Layout (pantalla 16:9, legible a 10 m)

- **Izquierda:** podio animado del top 5 (foto, nombre, puntos). Las posiciones se reordenan con animación.
- **Derecha:** carrera de barras con los 8 temas más hablados.
- **Arriba:** contadores grandes (conexiones, resúmenes, personas activas) y el QR para unirse.
- **Abajo:** feed de eventos recientes ("Ana ⇄ Luis hablaron de IA"), sin el contenido del resumen.
- Colores fuertes, tipografía enorme, transiciones de 300-500 ms.

Puntos

Acción	Puntos	Regla anti-spam
Conexión mutua nueva	+10	El mismo par puntúa una sola vez
Conexión con alguien de otra tribu ("puente")	+15 extra	Solo si ambos tienen tribu
Resumen aprobado	+25 por persona	La IA debe marcarlo como sustantivo
Primera conexión del día	+5	Una vez por persona

Tope: máximo 12 conexiones con puntos por persona por hora.

Temas en vivo

- Vocabulario fijo de ~40 temas de JS/IA (React, Node, TypeScript, Agentes, LLMs, DX, Testing, Diseño, Empleo, Startups, etc.) más "Otros".
- Antes de que existan resúmenes, cada conexión suma los intereses en común de la pareja ("temas que conectan").
- Cada resumen aprobado suma sus temas con peso ×3 ("temas que se platican").

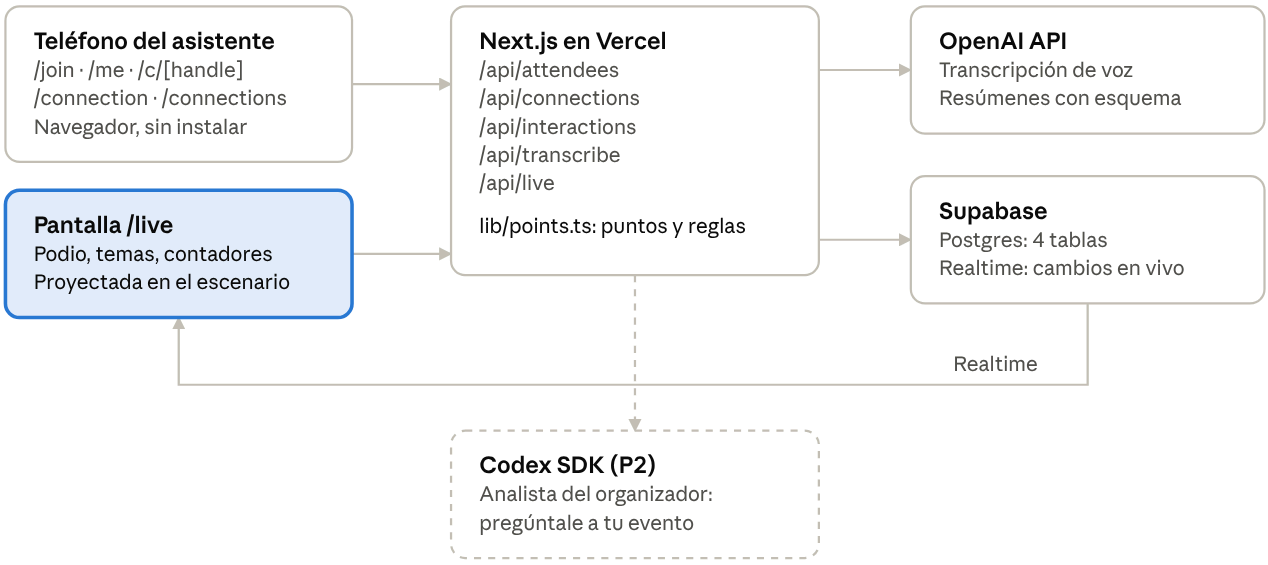
P2: rondas e insignias

- Rondas de 15-20 min que reinician el marcador; el ganador se anuncia en cada break.
- Insignias: Puente (3+ tribus distintas), Conversador (5+ resúmenes), Racha (3 conexiones en 10 min).
- Marcador de tribus vs. tribus.

Arquitectura y stack

Una sola app Next.js en Vercel sirve el teléfono y `/live`; Supabase guarda los datos y empuja los cambios en tiempo real, y OpenAI solo se llama desde el servidor.

Todo pasa por el servidor y /live escucha cambios en tiempo real



arquitectura · 5 componentes

Capa	Elección	Por qué
Frontend y API	Next.js (App Router) + TypeScript + Tailwind	Un repo, un deploy, el idioma de la audiencia de JSConf
Datos y tiempo real	Supabase (Postgres + Realtime)	Tablas en minutos y suscripciones para `/live`
IA	OpenAI API (transcripción + Structured Outputs)	Salida JSON validada por esquema
QR	`qrcode` para generar; cámara nativa para escanear	Sin escáner propio: el QR es una URL
Animaciones	framer-motion	Podio y barras que se reordenan
Validación	zod	Mismos esquemas en cliente y servidor
Deploy	Vercel + dominio corto	HTTPS para micrófono y cámara

Escanear con la cámara nativa evita el problema de iPhone: Safari no tiene `BarcodeDetector`, así que no construimos escáner dentro de la app.

Modelo de datos y API

Cuatro tablas en Postgres (Supabase) bastan para el MVP; `/live` se alimenta con Supabase Realtime sobre `connections`, `interactions` y `points`.

```
create table attendees (
  id uuid primary key default gen_random_uuid(),
  handle text unique not null,          -- 'ana'
  code text not null,                   -- '7K2', recuperación de sesión
  name text not null,
  photo_url text, company text, role text,
  tribe text,                           -- 'Frontend' | 'Backend' | 'IA' |
'Diseño' | 'Otro'
  topics text[] default '{}',           -- del vocabulario fijo
  seeking text, offering text,
  links jsonb default '{}',             -- {linkedin, github, x, instagram,
facebook, tiktok, web}
  is_seed boolean default false,        -- datos simulados
  created_at timestamptz default now()
);

create table connections (
  id uuid primary key default gen_random_uuid(),
  a_id uuid references attendees(id),
  b_id uuid references attendees(id),
  method text check (method in ('qr','tag')),
  shared_topics text[] default '{}',
  created_at timestamptz default now(),
  check (a_id <> b_id)
);
create unique index connections_pair_uq
  on connections (least(a_id,b_id), greatest(a_id,b_id));

create table interactions (
  id uuid primary key default gen_random_uuid(),
  connection_id uuid references connections(id),
  author_id uuid references attendees(id),
  recorded boolean default false,
  consent_ok boolean default false,
  transcript text,
  summary text,
  topics text[] default '{}',
  next_steps text[] default '{}',
```

```
  substantive boolean default false,
  approved_at timestamptz
);

create table points (
  id bigserial primary key,
  attendee_id uuid references attendees(id),
  reason text,          -- 'connection' | 'bridge' | 'summary' |
'first_of_day'
  amount int,
  round int default 1,
  ref_id uuid,          -- conexión o interacción que lo generó
(idempotencia)
  created_at timestamptz default now(),
  unique (attendee_id, reason, ref_id)
);
```

El índice único sobre la pareja ordenada hace que A→B y B→A sean la misma conexión.

API (Next.js route handlers)

Método	Ruta	Qué hace
POST	`/api/attendees`	Crea perfil, normaliza enlaces, genera handle y código, setea cookie
POST	`/api/session/recover`	Handle + código → cookie
GET	`/api/attendees/search?q=`	Autocompletado de @tag (P1)
POST	`/api/connections`	`{ targetHandle, method }` → crea conexión idempotente y otorga puntos
GET	`/api/connections`	Mis conexiones con tarjeta y estado del resumen
POST	`/api/transcribe`	Audio → texto
POST	`/api/interactions/draft`	Texto → `{summary, topics, next_steps, substantive}`
POST	`/api/interactions/:id/approve`	Aprueba, comparte con ambos y otorga puntos

Método	Ruta	Qué hace
GET	`/api/live`	Snapshot inicial de leaderboard, temas y contadores

Reglas de negocio en servidor (nunca en el cliente): puntos, idempotencia, tope por hora y validación de que el autor pertenece a la conexión.

IA: transcripción, resúmenes y temas

La IA hace tres trabajos acotados con salida estructurada; nunca decide puntos ni permisos.

Trabajo	Entrada	Salida	Cuándo
Transcribir	Audio (webm/m4a, máx. 2 min)	Texto	Dictar perfil o resumen, grabación
Pre-llenar perfil	Texto dictado	JSON de campos del perfil	Opción "Dictar mi perfil"
Redactar resumen	Texto o transcripción + nombres y temas de ambos	`{summary, topics, next_steps, substantive}`	Antes de aprobar

Esquema del resumen (Structured Outputs / JSON Schema)

```
{
  "type": "object",
  "additionalProperties": false,
  "required": ["summary", "topics", "next_steps", "substantive"],
  "properties": {
    "summary": { "type": "string", "description": "2-4 frases en español, en
tercera persona: de qué hablaron A y B" },
    "topics": { "type": "array", "maxItems": 4, "items": { "type": "string",
"enum": ["<vocabulario fijo>"] } },
    "next_steps": { "type": "array", "maxItems": 3, "items": { "type":
"string" } },
    "substantive": { "type": "boolean", "description": "false si el texto no
describe una conversación real" }
  }
}
```

Prompt del sistema (resumen)

```
Eres el asistente de Pasillo en una conferencia de JavaScript.
Recibes la nota de {author} sobre su plática con {other}.
Escribe un resumen breve y neutral que ambos puedan leer.
No inventes datos que no estén en la nota. No incluyas datos sensibles
(salud, dinero, temas personales); si aparecen, omítelos.
Clasifica los temas SOLO con el vocabulario dado.
Marca substantive=false si la nota está vacía, es spam o no describe una
conversación.
```

Modelos: usar el modelo de transcripción y el modelo de texto vigentes en la cuenta de OpenAI del equipo; confirmar los nombres en la consola antes de codificarlos y ponerlos en variables de entorno (`OPENAI_STT_MODEL`, `OPENAI_TEXT_MODEL`).

Fallbacks: si la transcripción falla, mostrar el campo de texto; si el resumen falla, guardar el texto del usuario tal cual como resumen.

Privacidad y consentimiento

La regla es simple: nada se graba sin que el otro acepte, nada se comparte sin aprobación, y el organizador solo ve agregados.

- **Grabar:** al tocar "Grabar", la otra persona recibe un aviso en su pantalla y debe aceptar con un toque. Sin aceptación no se graba. El audio se borra después de transcribir.

- **Resumen:** el autor lo revisa y aprueba antes de que llegue a ambos. Solo el autor puede editarlo.
- **Perfil:** solo se comparte lo que el usuario llenó. Ninguna red es obligatoria.
- **`/live`:** muestra nombres y puntos del leaderboard, y temas agregados. Nunca muestra el texto de un resumen.
- **Opt-out del leaderboard:** casilla en el perfil para aparecer como "Anónimo".
- **Datos simulados:** se marcan con `is_seed` y se pueden apagar con un toggle antes de la demo pública.
- **Aviso en `/join`:** una línea que explica qué se guarda y para qué, con casilla de aceptación.

División del equipo y PRs

Cuatro personas, cuatro ramas, cada una con su hilo de Codex en un worktree; el PR #0 desbloquea a todos y se mergea primero.

Persona	Área	Rama	Pantallas / rutas
A (lead)	Base, perfil, sesión, QR	`feat/profile-qr`	`/join`, `/me`, `/c/[handle]`, `/api/attendees`, `/api/session/recover`
B	Conexiones y puntos	`feat/connections`	`/connection/[id]`, `/connections`, `/api/connections`, @tag
C	IA y resúmenes	`feat/summaries`	`/connection/[id]/summary`, `/api/transcribe`, `/api/interactions/*`, grabación
D	Dashboard y datos	`feat/live`	`/live`, `/api/live`, seeds, deploy

PRs en orden de merge

PR	Dueño	Contenido	Depende de
#0 Scaffold	A	Next.js + TS + Tailwind, cliente de Supabase, migración SQL, vocabulario de temas, `AGENTS.md`, deploy en Vercel	—

PR	Dueño	Contenido	Depende de
#1 Perfil y QR	A	Formulario, normalización de enlaces, handle/código, cookie, QR, `/c/[handle]`	#0
#2 Conexiones	B	Conexión idempotente, tarjeta con "Seguir en…", lista, puntos y tope	#0 (usa mocks hasta #1)
#3 Resúmenes	C	Dictado/texto → borrador → aprobar → compartir, puntos de resumen	#0, #2
#4 Live	D	Podio, carrera de temas, contadores, Realtime, seeds de ~150 asistentes	#0
#5 @tag	B	Autocompletado y conexión por handle	#2
#6 Grabación	C	Consentimiento del otro + grabación + transcripción	#3
#7 Extras	D	Rondas, insignias, tribus, analista con Codex	#4

Reglas de trabajo

- Solo A toca la migración SQL y `lib/types.ts`; los demás piden cambios por chat.
- PRs pequeños; cada PR incluye un video o GIF de 10 s del flujo y marca los criterios de aceptación.
- Revisión cruzada en menos de 10 min: B revisa a A, C a B, D a C, A a D. Correr `/review` de Codex antes de pedir revisión.
- `main` siempre desplegable; si algo rompe la demo, se revierte primero y se arregla después.

Plantilla de PR

```
## Qué hace
<una línea>

## Cómo probar
1. ...

## Criterios de aceptación
- [ ] ...

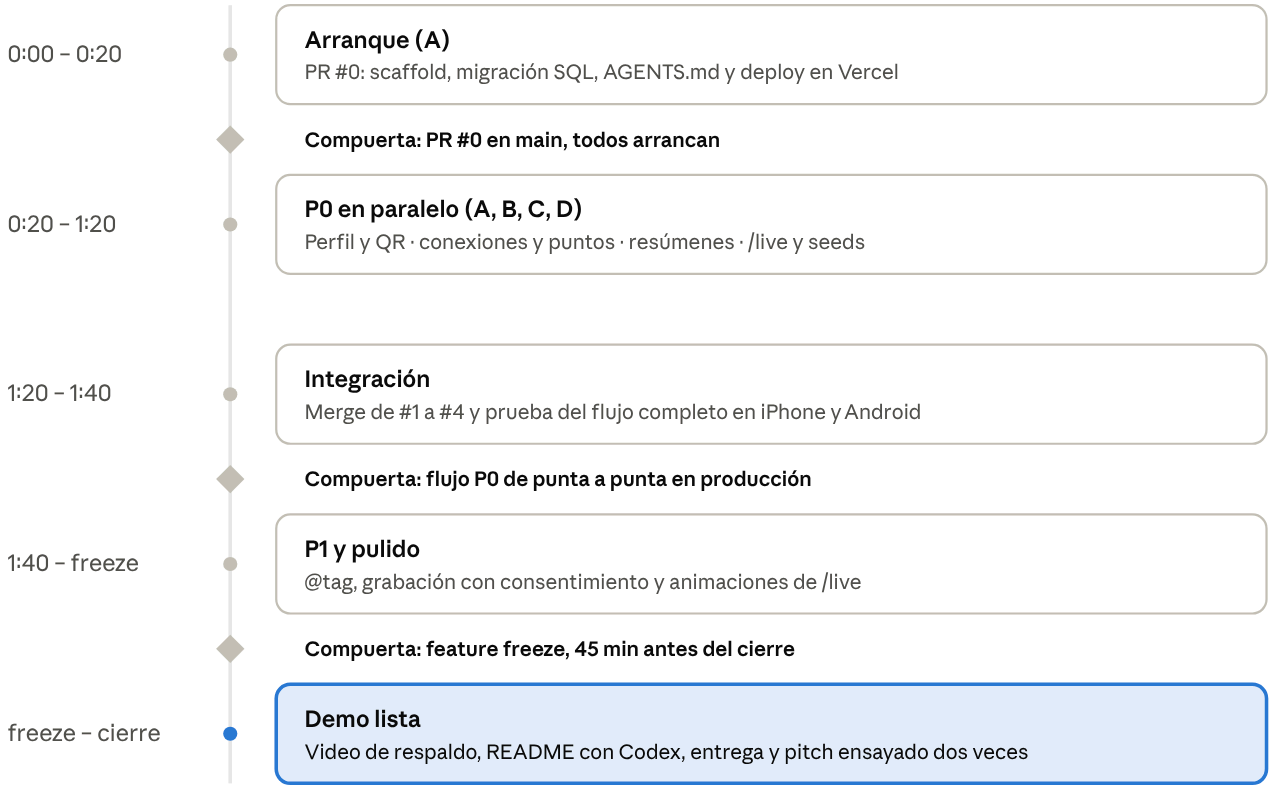
## Video / GIF
<enlace>

## Cómo usamos Codex
<hilo, worktree, skill o /review usados>
```

Cronograma

El plan usa tiempo relativo desde el arranque (T+0:00) y tres compuertas; nadie avanza de fase sin cruzar la anterior.

Tres compuertas ordenan la noche: scaffold, flujo completo y freeze



cronograma · 5 fases, 3 compuertas

Confirmar ya la hora de cierre. La página del evento dice 7 p. m. arriba y 9 p. m. en los detalles. Si es 7 p. m., el plan se reduce a P0 sin @tag ni grabación, con datos simulados y demo grabada.

AGENTS.md y prompts para Codex

El `AGENTS.md` es el contrato que comparten los cuatro hilos; cada persona pega su prompt en un hilo nuevo de Codex sobre un worktree de su rama.

`AGENTS.md` (raíz del repo, lo crea A en el PR #0)

```
# Pasillo — reglas para agentes

## Producto
Companion de networking para conferencias. Perfil una vez, conexión por QR o
@tag,
resumen aprobado que reciben ambos, dashboard /live estilo Kahoot. UI en
español.
```

```
## Stack
Next.js (App Router) + TypeScript estricto + Tailwind. Supabase (Postgres +
Realtime).
OpenAI vía server routes. Deploy en Vercel. pnpm.

## Contratos (no cambiar sin avisar al lead)
- Esquema: supabase/migrations/0001_init.sql
- Tipos: lib/types.ts
- Vocabulario de temas: lib/topics.ts
- Puntos: lib/points.ts (solo servidor)

## Reglas
- Nunca llamar a OpenAI ni otorgar puntos desde el cliente.
- Toda escritura valida con zod.
- Conexiones y puntos son idempotentes.
- Mobile first: probar a 375 px. /live se diseña para 1920x1080.
- No agregar login ni OAuth.
- Antes de terminar: pnpm lint && pnpm typecheck && pnpm build.

## Comandos
pnpm dev · pnpm lint · pnpm typecheck · pnpm build · pnpm seed
```

Prompt A — Scaffold, perfil y QR

```
Lee AGENTS.md. Crea el scaffold (Next.js App Router, TS, Tailwind, Supabase)
y la migración
0001_init.sql con las tablas attendees, connections, interactions y points
del documento.
Luego implementa /join (formulario con nombre obligatorio, redes opcionales
normalizadas a URL,
intereses del vocabulario, tribu), POST /api/attendees (handle único + código
de 3 caracteres,
cookie de sesión), /me con QR de https://<dominio>/c/<handle>, POST
/api/session/recover y
/c/[handle] (con sesión crea conexión vía /api/connections; sin sesión
muestra la tarjeta y
el formulario corto, y conecta al guardar). Agrega tests de la normalización
de enlaces.
```

Prompt B — Conexiones y puntos

```
Lee AGENTS.md. Implementa POST /api/connections { targetHandle, method }
idempotente usando el
índice único de la pareja, calcula shared_topics y otorga puntos con
lib/points.ts
(+10 conexión, +15 puente entre tribus, +5 primera del día, tope 12 por hora,
idempotente por ref_id).
Crea /connection/[id] con la tarjeta del otro y botones "Seguir en…" por red,
y /connections
con pendientes primero. Después, el @tag: GET /api/attendees/search y un
input con autocompletado.
Tests de idempotencia y del tope.
```

Prompt C — Resúmenes con IA

```
Lee AGENTS.md. Implementa POST /api/transcribe (audio de MediaRecorder, máx.
2 min),
POST /api/interactions/draft con Structured Outputs usando el esquema del
documento
(summary, topics del vocabulario, next_steps, substantive) y POST
/api/interactions/:id/approve
(valida que el autor sea parte de la conexión, bloquea si substantive=false,
otorga +25 a cada uno).
UI /connection/[id]/summary: dictar o escribir, ver borrador editable,
aprobar. Fallbacks:
sin transcripción → texto; sin IA → guardar el texto tal cual. Modelos por
variables de entorno.
```

Prompt D — Dashboard y datos

```
Lee AGENTS.md. Crea /live para 1920x1080: podio animado top 5 (framer-
motion), carrera de barras
de los 8 temas (temas en común de cada conexión x1, temas de resúmenes
aprobados x3), contadores,
QR para unirse y feed de eventos sin texto de resúmenes. Snapshot inicial con
GET /api/live y
actualización con Supabase Realtime. Crea pnpm seed: ~150 asistentes is_seed
con perfiles
realistas de una conferencia JS en Guadalajara, ~300 conexiones y ~80
resúmenes, más un
simulador opcional que agregue una conexión cada 3-5 s. Toggle para ocultar
los seeds.
```

Demo y pitch para JSConf

La demo dura 3 minutos y el público participa: escanean, se conectan entre ellos y ven `/live` moverse.

Tiempo	Qué pasa	En pantalla
0:00-0:20	"¿A cuántos conocieron hoy? ¿Recuerdan de qué hablaron?"	Título y problema
0:20-0:50	Todos escanean el QR y crean perfil (solo nombre)	`/live` con el contador subiendo
0:50-1:30	Pedir que se conecten con quien tengan al lado	Podio y temas moviéndose
1:30-2:10	Un miembro del equipo dicta un resumen en vivo y lo aprueba	Tarjeta, resumen y +25 en el podio
2:10-2:40	Vista del organizador: "esto es lo que le interesa a su comunidad"	Temas y contadores
2:40-3:00	Cómo usamos Codex (4 hilos, worktrees, AGENTS.md) y pedir el voto	QR + nombre del proyecto

Frases clave

- "Sin login, sin LinkedIn, en dos toques."
- "Ustedes acaban de generar los datos que ningún organizador tiene: de qué se habla en el pasillo."

Antes de subir: seeds encendidos, simulador encendido, `/live` en pantalla completa, video de respaldo abierto en otra pestaña.

Riesgos y plan B

El mayor riesgo es la demo en vivo (red, tiempo y adopción); cada riesgo tiene un plan B ya preparado.

Riesgo	Señal	Plan B
Wi-Fi del venue cae	`/live` no actualiza	Hotspot propio + simulador de seeds; video de respaldo
Poca gente se conecta en la demo	Contador < 10 a los 60 s	Simulador sube el ritmo; el equipo se conecta entre sí en vivo
iPhone pierde la sesión (Safari vs. otra app)	Usuario "desaparece"	Pantalla de recuperación con handle + código; no pedir instalar la app
Transcripción lenta o con ruido	> 5 s o texto vacío	Campo de texto siempre visible como alternativa
Resumen de IA falla	Error en `/draft`	Guardar el texto del usuario tal cual
Spam para subir en el podio	Muchas conexiones de una persona	Tope por hora y puntos solo por conexión mutua
Se acaba el tiempo	P0 incompleto a mitad de la noche	Recortar en el orden de la sección de alcance; congelar features 45 min antes del cierre
Hora de cierre incierta (la página dice 7 y 9 p. m.)	—	Confirmar con los organizadores ahora y ajustar el cronograma

Checklist de entrega

El proyecto está listo cuando el flujo completo funciona en producción desde dos teléfonos distintos.

- [ ] Flujo completo probado en iPhone (Safari) y Android (Chrome): perfil → QR → conexión → resumen → puntos → `/live`
- [ ] Desplegado en Vercel con dominio corto y QR impreso o en pantalla
- [ ] Seeds cargados y simulador funcionando; toggle para ocultarlos
- [ ] README: problema, demo en GIF, stack, cómo correrlo y sección "Cómo usamos Codex" (hilos, worktrees, AGENTS.md, `/review`, prompts)

- [ ] Video de respaldo de 60-90 s grabado
- [ ] Repositorio público con licencia
- [ ] Pitch de 3 min ensayado dos veces con cronómetro
- [ ] Formulario de entrega del hackathon enviado